THE ORACLE'S DECREE

Security Vulnerability Scanner for Modern Web Apps

Product Documentation & Technical Blueprint

Version 1.0	MVP Scope	March 2026

Table of Contents

1. Executive Summary
2. Product Overview & Vision
3. User Personas
4. System Architecture
5. Scan Engines — Deep Dive
6. GitHub Integration Flow
7. Manual File Upload Flow
8. User Journey & UX Guidance
9. Data Models
10. API Reference
11. Risk Scoring Algorithm
12. Workflow Diagrams
13. MVP Roadmap
14. Tech Stack

1. Executive Summary

The Oracle's Decree is a full-stack automated security intelligence platform built for modern web development teams. It scans source code, npm dependency trees, API specifications, and configuration files to detect real-time security risks — before a single line reaches production.

The platform unifies four scan engines (SAST, SCA, API Security, Config/Secrets) into a single prioritized risk score, with clear developer-facing fix instructions. Developers can trigger a scan in two ways: connecting their GitHub repository for automated scanning, or manually uploading files through a drag-and-drop interface.

> **Core Value Proposition**
> Most security tools give you a CVE list and call it done. The Oracle's Decree gives you a risk score, tells you exactly what is broken, why it matters, and the precise command to fix it — all in under 60 seconds.

1.1 Key Capabilities

- Dual input modes: GitHub repository connection + manual file upload
- Four integrated scan engines covering code, dependencies, APIs, and secrets
- Prioritized risk score (0–100) with CVSS-backed severity tiers
- Actionable fix suggestions — not just flags, but exact commands
- Live dashboard with vulnerability trends and developer-impact metrics
- Step-by-step in-app guidance so users never have to guess the next action
- Shareable scan reports (PDF + URL)

2. Product Overview & Vision

2.1 The Problem

Modern web apps are built on thousands of third-party packages, open APIs, and configuration files — each a potential entry point for attackers. In 2025 alone, supply-chain attacks compromised packages with over 2.6 billion combined weekly npm downloads. Despite this, most developer teams lack integrated tooling to catch these risks before shipping.

Pain Point	What Currently Happens
npm audit	Only catches known CVEs; misses behavioral/supply chain threats

SAST tools	Generate long CVE lists with no fix priority or developer routing
API scanners	Run separately, producing siloed reports nobody connects to code risk
Secret scanners	Often catch secrets after they're already pushed to git history
Developer UX	Security tools are complex — devs skip them or misread the output

2.2 Our Solution

The Oracle's Decree acts as an automated security co-pilot. It sits at the pre-production gate, receives code via GitHub webhook or file upload, runs all four scan engines in parallel, and presents the developer with a single, human-readable security brief in under 60 seconds.

2.3 Non-Goals (MVP)

- Real-time runtime monitoring (post-deployment)
- Mobile app scanning (iOS/Android)
- Binary or compiled artifact scanning
- Custom rule authoring (roadmap feature)

3. User Personas

Persona	Goals	Pain Points
Solo Developer	Ship quickly, avoid security surprises, no dedicated sec team	No time to learn complex tools, not sure which CVEs matter
Startup CTO	Protect product reputation, pass security audits, scale safely	Tools are expensive and require security expertise to interpret
Security Engineer	Automate pre-commit checks, reduce MTTR, track risk over time	Too many siloed tools with no unified risk view
Open-Source Maintainer	Ensure published packages are safe, build community trust	No automated gate before npm publish

4. System Architecture

4.1 Architecture Overview

The Oracle's Decree follows a modular, queue-based microservices architecture. Each scan engine runs as an independent worker, allowing parallel execution and independent scaling. The frontend communicates with a REST API gateway which orchestrates scan jobs and aggregates results.

> **Architecture Principle**
> All scan engines are stateless workers. They receive a job payload (file content + metadata) from a Redis queue and emit a structured finding report. This means any engine can be scaled horizontally without affecting others.

4.2 Component Map

Component	Responsibility
React Frontend	File upload UI, GitHub OAuth, scan dashboard, report viewer, in-app guidance system
API Gateway (Node.js)	Auth, request routing, job queuing, WebSocket for live progress
Job Queue (Redis + Bull)	Distributes scan jobs to engine workers, manages retry logic
SAST Engine	AST-based code pattern analysis for JS/TS/Python source files
SCA Engine	Parses package.json, queries OSV.dev + npm audit, behavioral flags
API Security Engine	OpenAPI/Swagger spec parser, OWASP API Top 10 rule checks
Config/Secrets Engine	Entropy analysis + pattern matching for secrets in config files
Aggregator Service	Collects engine results, computes risk score, generates report
PostgreSQL	Stores scan history, user accounts, project configs, finding records
GitHub App	OAuth, webhook receiver for push/PR events, repo file fetcher

4.3 Data Flow — High Level

The data flow begins when a developer either pushes code to a connected GitHub repository (triggering an automated webhook) or manually uploads files through the web interface. Both input paths converge at the API Gateway, which validates the input and enqueues a scan job.

1. Input received (GitHub webhook event OR manual file upload)
2. API Gateway authenticates the request and creates a Scan Job record in PostgreSQL

3. Job payload (file content + metadata) pushed to Redis queue
4. Available engine workers pull the job and process in parallel
5. Each engine emits a FindingSet JSON object back to the queue
6. Aggregator Service collects all FindingSet objects for the job
7. Risk score computed, report document generated
8. WebSocket notification sent to the frontend — scan complete
9. User sees full results dashboard

5. Scan Engines — Deep Dive

5.1 Engine 1 — SAST (Static Application Security Testing)

The SAST engine performs static analysis of source code files without executing them. It uses Abstract Syntax Tree (AST) parsing for deep structural analysis and regular expression pattern matching for fast surface-level checks.

Supported Languages

- JavaScript / TypeScript (via @babel/parser)
- Python (via tree-sitter-python)
- JSON configuration files

Detection Rules (MVP — 25 rules)

Rule ID	Description
SAST-001	eval() or new Function() usage — arbitrary code execution risk
SAST-002	document.innerHTML assignment — XSS sink
SAST-003	SQL string concatenation — SQL injection risk
SAST-004	Hardcoded credential patterns (password=, apiKey=, secret=)
SAST-005	Disabled TLS verification (rejectUnauthorized: false)
SAST-006	Insecure random (Math.random() for security-sensitive ops)
SAST-007	Unsafe deserialization (JSON.parse on untrusted input)
SAST-008	Missing input sanitization before DB query
SAST-009	console.log with sensitive variable names
SAST-010	Prototype pollution patterns (obj[key] = value without guard)

5.2 Engine 2 — SCA (Software Composition Analysis)

The SCA engine parses the project's dependency manifest (package.json and package-lock.json) and cross-references every dependency against multiple vulnerability databases. It also performs behavioral heuristic checks beyond CVE lookup.

Data Sources

- OSV.dev API — open-source vulnerability database (free, real-time)
- npm audit API — official npm security advisory feed
- GitHub Advisory Database — curated security advisories
- deps.dev API (Google) — dependency health metadata

Behavioral Heuristics (Supply Chain Defense)

- Package age < 7 days and high download count (typosquatting signal)
- Maintainer account age < 30 days (account takeover signal)
- Obfuscated postinstall scripts (base64-encoded shell commands)
- Sudden major version bump with new maintainer (suspicious transfer)
- Package name similarity to popular packages (Levenshtein distance < 2)

5.3 Engine 3 — API Security Scanner

The API security engine accepts an OpenAPI 3.0 / Swagger 2.0 specification (YAML or JSON) and checks every endpoint definition against OWASP API Security Top 10 rules.

OWASP API Top 10 Checks Implemented

10. **API1** Broken Object Level Authorization — checks for missing auth on resource endpoints
11. **API2** Broken Authentication — detects missing OAuth/API key requirements
12. **API3** Broken Object Property Level Authorization — identifies over-exposed response fields
13. **API4** Unrestricted Resource Consumption — flags missing rate limiting definitions
14. **API5** Broken Function Level Authorization — detects admin endpoints without elevated auth
15. **API6** Unrestricted Access to Sensitive Business Flows — no throttling on payment/OTP
16. **API7** Server Side Request Forgery — URL parameters that could trigger SSRF
17. **API8** Security Misconfiguration — permissive CORS, HTTP (non-HTTPS) server URLs

5.4 Engine 4 — Config & Secrets Scanner

The Config/Secrets engine scans non-source files for exposed credentials, insecure defaults, and sensitive configuration mistakes. It uses a combination of entropy analysis (Shannon entropy) and pattern matching.

File Types Scanned

- .env, .env.local, .env.production — environment variable files
- docker-compose.yml / Dockerfile — container configuration
- .github/workflows/*.yml — CI/CD pipeline definitions
- nginx.conf, apache.conf — web server configuration
- *.json, *.yaml, *.toml — generic config files

Detection Patterns

- AWS Access Keys (AKIA[0-9A-Z]{16})
- GitHub tokens (ghp_, gho_, ghs_ prefixes)
- Private key headers (-----BEGIN RSA PRIVATE KEY-----)
- Database connection strings with embedded passwords
- Stripe, Twilio, Sendgrid API key patterns
- DEBUG=true in production-named configs

6. GitHub Integration Flow

6.1 Overview

The GitHub integration allows developers to connect their repositories and receive automatic security scans on every push or pull request. The integration uses GitHub Apps (not OAuth Apps) for superior permission granularity and organization-level installation support.

6.2 Setup Flow — Step by Step

Guided Experience

Every step of the GitHub connection flow has inline tooltips, progress indicators, and error explanations. Users are never left wondering what to do next. If a step fails, the system explains why and offers a retry path.

1

Install the GitHub App

- User clicks 'Connect GitHub' on the dashboard
- Redirected to GitHub to install 'Oracle's Decree' GitHub App
- User selects 'All repositories' or specific repos to grant access
- GitHub redirects back with an installation_id
- System stores installation_id linked to user account

2

Authorize & Select Repository

- User sees a list of all repos from the installed GitHub App
- User selects the repository to scan
- System fetches repo metadata (default branch, language, file tree)
- User confirms — system configures the webhook automatically

3

Webhook Configuration (Automatic)

- System calls GitHub API to register a webhook on the selected repo
- Webhook listens for: push events, pull_request events
- A webhook secret is generated and stored encrypted in PostgreSQL
- System sends a test ping and confirms webhook is active

4

First Scan Triggered

- System immediately fetches the default branch file tree via GitHub API
- Files are filtered (source code, package.json, config files)
- Files submitted to all four scan engines in parallel
- User sees live progress on the dashboard (WebSocket updates)
- Scan complete notification shown — user directed to results

5

Ongoing Automatic Scans

- Every push to any watched branch triggers a new scan automatically
- Every pull request gets a scan — results posted as a PR check
- Developer sees pass/fail inline in the GitHub PR interface
- Scan history is preserved — trend charts updated

6.3 GitHub Pull Request Check Integration

When a pull request is opened or updated, Oracle's Decree runs a scan and posts the result as a GitHub Check. The PR shows a pass (green tick) or fail (red X) with a summary of findings. Developers can click through to the full report directly from GitHub.

Check Status	Condition
Passed (green)	Risk score below 30 — no critical or high severity findings
Warning (yellow)	Risk score 30–59 — medium findings present, no criticals
Failed (red)	Risk score 60+ OR any critical severity finding detected
Error (grey)	Scan engine error — system notifies user with retry option

7. Manual File Upload Flow

7.1 Overview

The manual upload mode is designed for developers who are not ready to connect a GitHub account, are working on private code, or want to do a one-off scan of specific files. The interface guides users through every step with clear instructions and live feedback.

7.2 Upload Flow — Step by Step

No Account Required for Trial

Users can perform one free manual scan without creating an account. This removes all friction for first-time users — they can experience the full value immediately.

1

Choose What to Scan

- User lands on Upload page — sees three clear scan targets to choose from
- Option A: Source Code Files (JS, TS, Python)
- Option B: package.json / package-lock.json only (dependency scan)
- Option C: API Specification (OpenAPI YAML/JSON)
- Option D: Upload everything — system auto-detects file types
- Inline tooltip explains what each option covers

2

Upload Files

- Drag-and-drop zone accepts files or entire folders (via directory upload)
- Files can also be selected via file picker
- System shows a live file tree as files are added
- Unsupported file types are highlighted with a tooltip explaining why
- User can remove files before scanning
- Max file size: 50MB per file, 200MB total

3

Configure Scan (Optional)

- User can enable/disable specific scan engines
- User can set the severity threshold for PR blocking (if using CI export)
- Advanced: user can paste a .env file for secrets scanning without uploading to server
- All optional — clicking 'Start Scan' with defaults works perfectly

4

Live Scan Progress

- Four progress bars — one per engine — show real-time status
- Each bar shows: Queued → Running → Complete with timing
- Partial results appear as engines complete (no waiting for all)
- Animated status indicators keep user engaged
- Estimated time remaining shown

5

View Results & Act

- Risk Score badge shown prominently (0–100 with color coding)
- Findings listed by severity: Critical → High → Medium → Low → Info
- Each finding has: file path, line number, description, fix command
- One-click copy for fix commands
- Export report as PDF or shareable link
- Option to re-scan after fixes are applied

7.3 Accepted File Types

Category	Accepted Extensions
JavaScript / TypeScript	.js, .jsx, .ts, .tsx, .mjs, .cjs
Python	.py
Package manifests	package.json, package-lock.json, yarn.lock, requirements.txt
API Specifications	.yaml, .yml, .json (OpenAPI/Swagger format)
Configuration files	.env, .env.*, docker-compose.yml, Dockerfile, *.conf, *.toml
CI/CD pipelines	.github/workflows/*.yml, .gitlab-ci.yml, Jenkinsfile

8. User Journey & In-App Guidance System

8.1 Design Principle

Oracle's Decree is built on the principle that security tools should never require security expertise to use. Every interaction is guided. Every result is explained. Every finding has a fix. The user should never need to Google anything — the system provides complete context inline.

8.2 Onboarding Flow (First-Time User)

18. User arrives at landing page — sees animated demo of a scan running
19. 'Try it free — no signup' CTA → taken directly to Upload page
20. Upload page shows a welcome tooltip: 'Drop your project files or connect GitHub to start'
21. User completes first scan — sees results
22. After results: 'Create a free account to save your scan history and set up GitHub monitoring'
23. Signup takes under 60 seconds (GitHub OAuth or email/password)

8.3 In-App Guidance Elements

Contextual Tooltips

Every form field, option, and result metric has an information icon. Hovering or tapping shows a plain-English explanation. Examples:

- Risk Score hover: 'This score combines severity, exploitability, and fix availability across all findings. Anything above 60 should be fixed before deploying.'
- CVSS Score hover: 'The Common Vulnerability Scoring System rates vulnerability severity on a 0–10 scale. 9–10 = Critical. 7–8.9 = High.'

Inline Fix Instructions

Every finding card contains a Fix section with:

- Plain-English explanation of why this is dangerous
- The exact terminal command to apply the fix
- A code example showing the secure alternative
- A link to the relevant CVE or OWASP page for deeper reading

Progress Guidance

When a scan is running, the system shows:

- Which engines are running and what they are checking
- 'Did you know?' security tips while waiting
- Partial results as soon as any engine completes

Empty States

If no findings are returned for a category, the system shows a positive confirmation: 'No dependency vulnerabilities found. Your packages are up to date.' — not just a blank panel.

8.4 Error Handling & Recovery Guidance

Error Scenario	What the System Shows
GitHub connection fails	'Your GitHub token may have expired. Click here to reconnect — takes 30 seconds.'
Unsupported file uploaded	File highlighted in red with tooltip: 'We support JS, TS, and Python files. [See full list]'
Scan engine timeout	'The SAST engine took too long on this file. We skipped it — here's why large files slow things down.'
Rate limit hit (OSV API)	'We hit a rate limit on the vulnerability database. Your scan will resume automatically in 15 seconds.'
No package.json found	'We couldn't find a package manifest. Drag in your package.json to enable dependency scanning.'

9. Data Models

9.1 Core Entities

User

Field	Type & Notes
id	UUID — primary key
email	string — unique, indexed
github_installation_id	string | null — GitHub App installation
github_token_encrypted	string | null — encrypted OAuth token
plan	enum: free | pro | team
created_at	timestamp

Project

Field	Type & Notes

id	UUID — primary key
user_id	FK → User
name	string
github_repo_full_name	string | null — e.g. 'owner/repo'
github_webhook_id	string | null
default_branch	string — default 'main'
scan_on_push	boolean
block_pr_threshold	integer — risk score threshold for PR block

Scan

Field	Type & Notes
id	UUID — primary key
project_id	FK → Project
trigger	enum: manual | github_push | github_pr
status	enum: queued | running | complete | failed
risk_score	integer 0–100
findings_critical	integer — count
findings_high	integer — count
findings_medium	integer — count
findings_low	integer — count
github_commit_sha	string | null
github_check_run_id	string | null
started_at	timestamp
completed_at	timestamp | null

Finding

Field	Type & Notes
id	UUID — primary key
scan_id	FK → Scan
engine	enum: sast | sca | api | secrets
rule_id	string — e.g. 'SAST-001'

severity	enum: critical | high | medium | low | info
cvss_score	float | null
title	string
description	string
file_path	string | null
line_number	integer | null
package_name	string | null — for SCA findings
fix_command	string | null — exact terminal command
fix_description	string
cve_id	string | null
owasp_category	string | null

10. API Reference

10.1 Authentication

All API endpoints (except /auth/*) require a Bearer token in the Authorization header. Tokens are issued on login and expire after 24 hours.

> **Base URL**
> https://api.oracles-decree.com/v1 (production) https://api-staging.oracles-decree.com/v1 (staging)

10.2 Core Endpoints

Method + Path	Description
POST /auth/login	Email/password login — returns JWT
GET /auth/github/callback	GitHub OAuth callback — issues JWT
GET /projects	List all projects for authenticated user
POST /projects	Create a new project
GET /projects/:id/scans	List scan history for a project
POST /scans/upload	Submit files for manual scan (multipart/form-data)

GET /scans/:id	Get scan status and results
GET /scans/:id/report	Download PDF report
GET /scans/:id/findings	List all findings with pagination
POST /github/webhook	GitHub webhook receiver (internal use)
GET /github/repos	List repos available from GitHub App installation
POST /github/connect	Connect a repo and configure webhook

10.3 Scan Upload Request

```
POST /scans/upload
Content-Type: multipart/form-data

Body fields:
  files[]        — one or more source files
  project_id     — UUID (optional, links to project)
  engines[]      — array: sast,sca,api,secrets (defaults all)
  severity_min   — minimum severity to report (default: low)
```

11. Risk Scoring Algorithm

11.1 Score Composition

The overall risk score is a weighted composite of findings across all four engines. The score ranges from 0 (clean) to 100 (critical exposure). It is not a simple average — severity and exploitability are weighted heavily.

Factor	Weight in Score
Critical severity findings	25 points each (capped at 75)
High severity findings	10 points each (capped at 40)
Medium severity findings	3 points each (capped at 15)
Low severity findings	0.5 points each (capped at 5)
Supply chain behavioral flags	+10 bonus per flag (uncapped)
Secrets found in config	+20 per confirmed secret
Fix available reduction	−5 for each critical with known fix

11.2 Score Tiers

Score Range	Tier & Meaning
0–29	Low Risk — safe to deploy with minor recommendations
30–59	Medium Risk — review and fix highs before production deploy
60–79	High Risk — block deploy; fix criticals and highs immediately
80–100	Critical Risk — immediate remediation required; do not deploy

12. Workflow Diagrams

The following workflow diagrams are described in detail below. Visual SVG/PNG versions are embedded in the companion interactive documentation and the dashboard's 'How it works' overlay.

12.1 GitHub Integration Workflow

Step	Actor & Action
1	Developer: Clicks 'Connect GitHub' in Oracle's Decree dashboard
2	System: Redirects to GitHub — GitHub App installation consent screen
3	Developer: Grants repo access, redirected back with installation_id
4	System: Stores installation_id, calls GitHub API to list repos
5	Developer: Selects repository to monitor
6	System: Registers webhook on selected repo via GitHub API
7	System: Fetches current default branch file tree for initial scan
8	System: Enqueues scan job — all 4 engines run in parallel
9	Developer: Views live scan progress on dashboard
10	System: Scan complete — risk score computed, report stored
11	Developer: Reviews findings, applies fixes
12	Developer: Pushes new code → webhook fires → steps 8–11 repeat automatically

12.2 Manual Upload Workflow

Step	Actor & Action
1	Developer: Navigates to Upload page
2	Developer: Selects scan type (code/deps/API/all)
3	Developer: Drags files into upload zone or uses file picker
4	System: Validates file types, shows file tree preview
5	Developer: Optionally configures which engines to run
6	Developer: Clicks 'Start Scan'
7	System: Files uploaded securely to temporary storage (deleted post-scan)
8	System: Scan job enqueued — partial results streamed via WebSocket
9	Developer: Watches live progress bars per engine
10	System: All engines complete — risk score shown
11	Developer: Reviews findings — copies fix commands
12	Developer: Optionally exports PDF report or shareable link

12.3 Scan Engine Parallel Processing Workflow

Stage	Detail
Input Routing	API Gateway inspects uploaded files and routes each to relevant engines. A .py file goes to SAST. package.json goes to SCA. openapi.yaml goes to API engine. .env goes to secrets engine.
Parallel Execution	All relevant engines start simultaneously. Each engine processes independently. Estimated time: SAST 5–30s, SCA 3–15s, API 2–8s, Secrets 1–5s.
Result Aggregation	Aggregator collects FindingSet from each engine as they complete. Risk score recalculated after each engine reports.
Report Generation	Final report assembled — findings sorted by severity. PR check updated (GitHub mode). WebSocket notification sent to user.

13. MVP Roadmap

Phase 1 — Core (Weeks 1–2)

- File upload UI with drag-and-drop and file type validation
- package.json SCA engine — OSV.dev + npm audit integration
- 20 SAST regex rules for JS/TS
- Basic risk score calculator
- Results dashboard — finding cards with severity badges
- User accounts (email/password)

Phase 2 — GitHub Integration (Weeks 3–4)

- GitHub App creation and OAuth flow
- Webhook receiver and automatic scan triggering
- GitHub PR Check API integration
- Repository selector and project management
- Scan history and trend charts
- WebSocket live scan progress

Phase 3 — Advanced Engines (Weeks 5–6)

- API Security engine (OpenAPI spec parser + OWASP rules)
- Config/Secrets engine (entropy analysis + pattern matching)
- Supply chain behavioral heuristics
- PDF report export
- Shareable scan report links
- In-app guidance system (tooltips, contextual help)

Phase 4 — Polish & Growth (Weeks 7–8)

- Fix suggestion engine — one-click copy commands
- Developer attribution (which team member introduced which finding)
- Email notifications for critical findings
- Team/org accounts and shared project access
- Public API for CI/CD integration (GitHub Actions action)

14. Tech Stack

Layer	Technology
Frontend	React 18, Tailwind CSS, Zustand (state), React Query, Chart.js
Backend API	Node.js 20, Fastify, Zod (validation), Prisma ORM
Job Queue	Redis 7, Bull (queue library)
Database	PostgreSQL 16
SAST Parsing	@babel/parser (JS/TS), tree-sitter (Python)
SCA Data	OSV.dev API, npm audit CLI, GitHub Advisory GraphQL API
Auth	JWT (access tokens), GitHub OAuth App, bcrypt
File Handling	Multipart upload via @fastify/multipart, temporary S3 storage
WebSockets	ws library via Fastify plugin
GitHub Integration	Octokit REST + GraphQL, GitHub Apps (not OAuth Apps)
PDF Reports	Puppeteer (headless Chrome HTML-to-PDF)
Hosting (MVP)	Railway.app (API + workers), Vercel (frontend), Upstash (Redis)
Monitoring	Sentry (errors), Posthog (analytics)

Oracle's Decree — Security for Every Developer

This document is confidential and intended for internal use and technical stakeholders only.

Version 1.0 | March 2026